Enhanced SPECTRA – Revision Performance Evaluation

Experimental protocol (tests T1–T8)

Version 8 – 21 October 2026 (results of T1–T8)

1 Purpose and scope

This document defines the performance evaluation campaign executed for the major revision of the manuscript. All measurements are collected on the revised chaincode implementation; results obtained with earlier chaincode versions or with the former CLI/HTTP-based evaluation path are not reused.

The campaign is deliberately limited to the experiments required to answer the reviewers' performance-related comments (Table 1). Each test is designed to close several comments at once. Aspects that cannot be evaluated with the available infrastructure are reported as explicit limitations (Section 5) rather than approximated.

2 Common methodology

2.1 System under test

- Hyperledger Fabric 2.5.10 (Fabric `test-network`); two organisations (FarmersMSP, ProducersMSP), one peer each; single-node Raft ordering service (`etcdraft`, one consenter); channel `milkchannel`; state database LevelDB; TLS enabled; one Fabric CA per organisation.
- Chaincodes `certificate` (contracts `OriginContract`, `ProcessContract`, `PackagingCertContract`) and `userauth` (`UserAuthContract`), Node.js, version 1.0, sequence 1; SHA-256 of the published sources (comments removed, functionally identical to the deployed version): `certificate.js 074629aaa06faee0a0ad63245586f48f926a6d3894119759645d54e9b1489b16`, `userauth.js 9a364c0204786c3dcace7136d98da9fd42a2476c4b915b2701a3833bb9362726`. Endorsement policy: channel default `/Channel/Application/Endorsement` (`MAJORITY`), i.e. endorsement by *both* organisations. Every protected operation of `certificate` calls `UserAuthContract:CheckAuthorized` through `invokeChaincode`.
- Private data collection `uidPdc`: members FarmersMSP and ProducersMSP, `requiredPeerCount`=1, `maxPeerCount`=2, `blockToLive`=0 (never purged), member-only read and write.
- Block cutting parameters (read from the live channel configuration): `BatchTimeout` = 2 s, `MaxMessageCount` = 10, `PreferredMaxBytes` = 512 KB, `AbsoluteMaxBytes` = 99 MB. These values are recorded because they dominate the latency of state-changing transactions at low load.
- Host: Intel Core Ultra 7 155H, 32 GB RAM, NVMe SSD, Windows 11 Enterprise; WSL2 (kernel 5.15.167.4) with 22 logical CPUs and 15 GiB RAM assigned; Docker Engine 24.0.7 inside WSL2; Node.js 22.20.0. Host on AC power, sleep disabled during measurements.
- Client identities, registered and approved in `userauth` before the campaign: `bench-farmer` (User1, FarmersMSP, role farmer), `bench-producer` (User1, ProducersMSP, role producer),

`bench-consumer` (User2, ProducersMSP, role consumer). The UID pepper is initialised once with 32 random bytes passed as transient data.

2.2 Operations under test

Class	Operation	Assets required
Read-only	`OriginContract:GetProductionByLotCode`	existing lot
	`PackagingCertContract:GetCert`	existing certificate
	`PackagingCertContract:VerifyCertIntegrity`	existing certificate
	`UserAuthContract:CheckAuthorized`	registered identity
	`PackagingCertContract:VerifyByUID`	bound UID
State-changing	`OriginContract:RegisterProduction`	none
	`ProcessContract:RegisterTransformation`	available input lot
	`PackagingCertContract:CreatePackageForLot`	transformed lot
	`PackagingCertContract:RegisterBindingForPackage`	package, fresh UID
	`PackagingCertContract:ConsumeByUID`	bound, valid UID

Contention analysis (from the chaincode source). Lot, transformation, package and certificate identifiers are derived from the transaction ID (`LOT_CODE_MODE = 'TXID'`); the global lot counter is not used. Every state-changing operation writes only keys of the assets it operates on, and availability is tracked with one key per lot (`avail::<family>::<lot>`). The only keys read by all transactions (the pepper in `uidPdc` and the caller's record in `userauth`) are never written during the workload. Consequently, operations on distinct assets do not conflict under MVCC validation, and all state-changing operations can be used in the concurrent load tests. Conflicts arise only when the same asset is targeted concurrently, which is the subject of T3.

2.3 Load generation

- **Harness**: a dedicated Node.js client based on `@hyperledger/fabric-gateway` 1.7, used for all tests. It executes a predefined submission schedule (closed-loop in T1, open-loop with fixed or precomputed inter-arrival times in T2 and T4) and records per-stage timestamps with a monotonic clock (`performance.now()`). Each identity uses up to 32 independent gRPC connections. The same client is used for the latency decomposition and for the load tests, so that all results share the same measurement boundaries; Caliper, initially planned for T2, was not needed.
- Asset preparation, warm-up and post-run verification are always excluded from the measured interval.

2.4 Measurement boundaries

For a state-changing transaction i submitted through the Fabric Gateway, four timestamps are recorded:

- t_0: proposal creation by the client;
- t_1: return of `endorse()` (proposal sent, chaincode executed and endorsements collected from the required organisations);
- t_2: return of `submit()` (endorsed transaction accepted by the ordering service);
- t_3: return of `getStatus()` (commit event received with the validation code).

The latency components are

$$L_i^{\text{end}} = t_1 - t_0, \qquad L_i^{\text{sub}} = t_2 - t_1, \qquad L_i^{\text{com}} = t_3 - t_2, \qquad L_i^{\text{E2E}} = t_3 - t_0. \tag{1}$$

L^{com} includes block cutting, block dissemination, validation and commit. For read-only operations a single interval $L_i^{\text{E2E}} = t_{\text{resp}} - t_0$ is measured around `evaluate()`. All latencies are client-observed and exclude the web application, which is not part of the measured path.

2.5 Throughput, goodput and saturation

$$\lambda = \frac{N_{\text{submitted}}}{T_{\text{submit}}}, \qquad X = \frac{N_{\text{completed}}}{T_{\text{obs}}}, \qquad G = \frac{N_{\text{valid}}}{T_{\text{obs}}}, \tag{2}$$

where λ is the measured (not configured) offered rate, X the achieved throughput, G the goodput (transactions committed with validation code `VALID` and passing the application-level checks) and T_{obs} the measurement window. Reported throughput values are always *averages over the measurement window*; peak values are not used.

Saturation criterion (declared before the experiments). An offered rate λ is *sustainable* if, in every repetition: (i) $G \geq 0.95\,\lambda$; (ii) the failure rate is $\leq 1\%$; (iii) the P95 latency does not exceed twice the P95 measured at the lowest tested rate; (iv) the number of outstanding transactions does not grow during the measurement window. The *saturation point* is the lowest tested rate violating at least one condition, and the *maximum sustainable throughput* is the goodput at the highest sustainable rate.

2.6 Failure classification

Every non-successful request is assigned to one category: endorsement failure; chaincode/application rejection (expected or unexpected); MVCC read conflict; other invalidation at commit; client timeout; connectivity error; unknown outcome. Unknown outcomes are reconciled against the ledger by transaction ID before classification. Correct rejections (e.g. a second consumption of the same certificate) are reported as expected outcomes, not as failures.

2.7 Resource and storage monitoring

A sampler records `docker stats` for all Fabric and chaincode containers and host CPU/memory at 1 s intervals, together with timestamps aligned with the client logs. Storage is measured with `du` inside the peer containers, separately for block files (`chains`), state database (`stateLeveldb`), private data store (`pvtdataStore`) and history database (`historyLeveldb`), and for the orderer block files, before and after each workload phase. The orderer Raft write-ahead log (`etcdraft`, about 122 MB pre-allocated at start-up) is excluded because its size does not reflect ledger growth.

2.8 Repetitions and statistics

Unless stated otherwise, each configuration is repeated $R = 5$ times. For each run we report mean, median, standard deviation, P95, P99, minimum and maximum. Across runs we report the mean of run-level means with its 95% confidence interval (Student's t, $R - 1$ degrees of freedom) and the between-run coefficient of variation. Pooled percentiles are computed on the pooled sample, never by averaging run-level percentiles. No run is excluded unless a documented infrastructure or measurement fault occurred.

2.9 Data organisation

Each run produces one CSV file with one row per request (columns `test`, `config`, `run`, `req_id`, `op`, `client`, `t0`, `t1`, `t2`, `t3`, `tx_id`, `status`, `failure_class`), the resource samples, the storage snapshots and a manifest with environment and configuration identifiers. Directory scheme: `results/T<n>/<config>/r<k>/`.

3 Tests

3.1 T1 – Per-operation latency and decomposition

Question. What is the latency of each operation, and how is it distributed among endorsement (including chaincode execution), submission to the ordering service, and block cutting/commit?

Design. Sequential closed-loop execution with one outstanding request at a time. Each iteration executes the complete workflow on fresh assets: `RegisterProduction` (10 kg), `RegisterTransformation` (EXTRACTION), `GetProductionByLotCode`, `CreatePackageForLot`, `RegisterBindingForPackage` (fresh random 7-byte UID), `GetCert`, `VerifyCertIntegrity`, `CheckAuthorized`, `VerifyByUID`, `ConsumeByUID` and a second `ConsumeByUID` on the same UID (repeated scan, expected outcome `ALREADY_CONSUMED`). Every response is validated at application level (mass balance $10 \rightarrow 3 + 7$ kg, net package quantity 2.98 kg, certificate states). 3 warm-up iterations (excluded) and 60 measured iterations per run, $R = 5$ runs, i.e. 300 measurements per operation. The ledger is not reset between runs. Resource usage (`docker stats`, $\approx$1 s) and ledger storage before and after each run are recorded.

T1-b (causal check). The complete T1 workflow is repeated with `BatchTimeout` $\in \{1\,\text{s}, 0.5\,\text{s}\}$ (signed channel configuration update, all other parameters unchanged; 2 warm-up and 30 measured iterations, $R = 5$) and compared with T1 (2 s). The default value is restored afterwards.

Metrics. $L^{\text{end}}, L^{\text{sub}}, L^{\text{com}}, L^{\text{E2E}}$ statistics; CPU per container.

Outputs. Table of per-operation latency; stacked-bar figure of the latency components; table of T1-b.

Interpretation constraints. Differences in L^{end} between operations estimate the additional chaincode execution cost of the Enhanced SPECTRA mechanisms (mass balance, binding with PDC and HMAC, single-use consumption). Differences in L^{E2E} alone are not used for this purpose.

3.2 T2 – Offered load, saturation and storage growth

Question. How do throughput and latency evolve with the offered rate, where does the system saturate, and how much persistent storage does each operation consume?

Design. Open-loop fixed-rate load generated by the harness (32 connections per identity). Workloads: `RegisterProduction` (state-changing, independent keys, includes the `userauth` authorization call) and `VerifyByUID` (read-only, on 50 prepared bound UIDs). Offered rates $\lambda \in \{10, 20, 40, 80, 160, 320, 640, 1280\}$ TPS (first session), refined with intermediate rates in a second session on a freshly re-created network (writes 160–240 TPS, reads 320–400 TPS), repeating one rate of the first session as an anchor. Each level: 15 s warm-up, 30 s measurement, up to 30 s drain; $R = 3$. A sweep stops after two consecutive non-sustainable levels. In the second session the P95 reference of the saturation criterion is the value measured at the lowest rate of the first session.

Storage. Volume snapshots before and after asset preparation and after each level give the storage increment per operation type,

$$s_{\text{op}} = \frac{S_{\text{after}} - S_{\text{before}}}{N_{\text{valid,op}}}, \tag{3}$$

separately for blocks, state database and private data.

Metrics. λ, X, G, P50/P95/P99 latency, failure classes, outstanding requests, CPU/memory time series, storage.

Outputs. Throughput-vs-offered-load and latency-vs-offered-load curves; saturation point and maximum sustainable throughput; resource utilisation at saturation; storage-per-operation table and storage growth curve.

3.3 T3 – Concurrent conflicting operations

Question. Do the security invariants hold when conflicting operations are submitted concurrently?

Design. $k \in \{2, 5, 10, 20\}$ clients, each with its own gRPC connection, submit the same conflicting operation simultaneously (released together; the observed release spread is reported with the results). 10 targets per k and repetition, $R = 3$ (30 targets per k and sub-test). Target assets are prepared beforehand by parallel workflows.

- T3-a: `ConsumeByUID` on the same UID (single consumption, Eq. 16 of the manuscript);
- T3-b: `RegisterBindingForPackage` of the same UID on different packages, and of different UIDs on the same package (active binding uniqueness, Eqs. 13–14);
- T3-c: `RegisterTransformation` on the same input lot (input-consumption integrity).

Expected outcome (declared in advance). Exactly one valid committed state transition per target; every other request is either rejected by the chaincode or invalidated with `MVCC_READ_CONFLICT`. For T3-a, a request endorsed after the consumption has been committed reads the `CONSUMED` state and is committed as a valid repeated scan (`ALREADY_CONSUMED`, scan counter incremented); it is therefore classified separately from the single consumption. The final state is verified by independent queries.

Outputs. Table: k, targets, valid commits per target, distribution of rejection causes, invariant violations (expected: 0).

3.4 T4 – Transactional bursts

Question. How does the system absorb load peaks and how fast does it return to its baseline?

Design. Precomputed submission schedules (saved with their random seed) on the T2 state-changing operation:

- Profile A, single burst: $B \in \{10, 50, 100, 200\}$ requests released at the same instant (target spread within 100 ms; the observed spread is reported with the results);
- Profile B, periodic bursts: $B = 50$ every 5 s for 60 s;
- Profile C, Poisson arrivals: exponential inter-arrival times with mean rates of 80 and 128 TPS (50% and 80% of the maximum sustainable state-changing rate of 160 TPS measured in T2), 60 s per run, seed 20261021.

$R = 5$ per configuration, 10 s of idle time between runs, 32 client connections.

Metrics. Burst completion time (first submission to last commit), drain time, maximum outstanding requests, latency distribution, failure classes, recovery to baseline latency.

Outputs. Burst response table; outstanding-requests timeline for profiles B and C.

3.5 T5 – Fault injection and recovery

Question. How do component failures affect availability, and is the ledger state consistent after recovery?

Design. Two concurrent open-loop streams at 20% of the maximum sustainable rates of T2, both submitted through the FarmersMSP gateway peer: `RegisterProduction` at 32 TPS and `VerifyByUID` at 64 TPS (on 20 prepared bound UIDs). Timeline per run: 60 s baseline, 30 s fault, restoration, 90 s observation. Faults:

- F1: stop of `peer0.producers` (`docker stop/start`);
- F2: stop of the ordering service;
- F3: termination of the `certificate` chaincode container of FarmersMSP;
- F4: network partition of `peer0.producers` (`docker network disconnect/connect` from all the Docker networks the peer is attached to; a first series that disconnected only one of the two networks is reported as F4p).

$R = 3$ per fault.

Metrics. Per-second success/failure timeline per stream; outage duration; recovery time $T_{\text{rec}} = t_{\text{first valid commit after restore}} - t_{\text{restore}}$; failure classes; post-recovery reconciliation of all submitted transaction IDs (committed, invalidated, not committed) and check of the supply-chain invariants (no duplicate consumption, no lost or duplicated state transitions).

Interpretation constraints. With a single peer per organisation and an endorsement policy requiring both organisations, the expected behaviour under F1 is unavailability of state-changing operations; with a single orderer, F2 interrupts ordering. These outcomes characterise the availability of the tested topology and are not presented as fault tolerance of Fabric in general.

Outputs. Availability timelines; recovery-time table; reconciliation report.

3.6 T6 – Emulated network conditions

Question. How sensitive are latency and throughput to inter-node network delay, jitter and packet loss?

Design. Planned: `tc netem` applied to the egress interface of every Fabric container, RTT $\in \{\text{baseline}, 20, 50, 100\}$ ms, plus jitter and packet loss. Feasibility check (Step 0): the WSL2 kernel does not provide the `netem` queueing discipline ("qdisc kind is unknown"). Adopted method: analytical estimate of the effect of the inter-site RTT, adding the network delay on the critical path of each transaction type to the latencies measured in T1 and T2; the geographically distributed deployment is declared as a limitation.

Outputs. Latency components vs RTT; goodput and failure rate vs network condition.

Interpretation constraints. Results are reported as network emulation on a single host and not as a geographically distributed deployment.

3.7 T7 (optional) – Comparison with the original SPECTRA

Planned as an experimental comparison on the same network. Adopted method: functional comparison of the mechanisms of SPECTRA and Enhanced SPECTRA, reporting for each new mechanism its chaincode execution cost (endorsement phase measured in T1) and the concurrency evidence of T3, since end-to-end latencies of both implementations are dominated by block cutting.

3.8 T8 – Effect of supply-chain complexity

Question. How do the cost of the mechanisms, the verification latency and the certificate size evolve with the structure of the supply chain, i.e. with the number of lots merged by a transformation (breadth) and with the number of successive transformations (depth)?

Design. Sequential closed-loop submission, as in T1, so that the endorsement phase isolates chaincode execution. Part A (breadth): one EXTRACTION transformation with $n \in \{1, 5, 10, 25, 50, 100\}$ input lots (10 kg each; expected output $3n$ kg, checked at application level), followed by packaging, binding, 5 UID verifications, certificate query and consumption; 3 runs $\times$ 8 iterations per n, plus 1 excluded warm-up iteration per run. Part B (depth): chains of $d \in \{1, 5, 10, 20\}$ successive DRYING transformations (yield 0.95 per step) followed by the same steps; 5 chains per d plus 1 excluded warm-up chain. Input lots are prepared in parallel and are not part of the measurements. Suggested by an internal review and by the request of R2 for experiments involving complex supply-chain transformations.

Outputs. Chaincode execution of each step, UID verification latency and certificate size as a function of n and d; linear fits.

Reviewer	Comment	Addressed by
R1	Separate blockchain processing from application overhead; latency breakdown	T1, T1-b
R1	Burst duration and inter-arrival distribution not specified	T4
R1	Peak or average throughput?	Sec. 2.5 (definition), T2
R1	"Saturation" not defined	Sec. 2.5 (criterion), T2
R1	Units (wt/s, wf/s)	All throughput values in TPS
R1	Evaluation only local, no distributed latency	T6 (analytical estimate), limitations
R1	bindingId and cloned UIDs, no validation of resistance	T3-a, T3-b
R2	Single laptop, loopback, no realistic network delay	T6 (analytical estimate), limitations
R2	Resource consumption, storage growth	T2, monitoring in all tests
R2	Endorsement/orderer overhead, transaction finality	T1, T1-b
R2	Failure-recovery tests, peer failures, network partitions	T5
R2	Informal security analysis	T3 (experimental check of invariants)
R2	Scalability claims not supported	T2, moderated claims
R2	Comparison with original SPECTRA	T7 (functional comparison)
R2	Complex supply-chain transformations, branched supply chains	T8
R2	Median > mean, inconsistent throughput values	Former evaluation replaced
R2	Reproducibility (code, configs, workloads, raw data)	Sec. 2.9, repository / supplementary material

Table 1: Mapping between reviewer comments and the evidence produced by the campaign.

4 Traceability to the reviewers' comments

5 Explicit limitations

- No geographically distributed or heterogeneous multi-host deployment; the effect of network delay is estimated analytically (T6), since network emulation was not available on the host.
- One peer per organisation: no peer redundancy is evaluated.
- No malicious peers or Byzantine behaviour; the ordering service is crash-fault tolerant at most.
- Long-term ledger growth is extrapolated from the measured per-operation storage cost, not observed over months of operation.
- Supply-chain structures are evaluated up to 100 input lots per transformation and 20 successive transformations (T8); arbitrary transformation graphs were not reproduced.
- Privacy leakage through metadata is analysed qualitatively in the security section, not measured.

6 Results

6.1 T1 – Per-operation latency and decomposition

Executed on 20 October 2026 (14:49–15:55 CEST), configuration `bt2s` (`BatchTimeout` = 2 s): 5 runs × 63 iterations (3 warm-up iterations excluded), 3,300 measured requests. All requests completed successfully and passed the application-level checks; no run was excluded. Results are reported in Tables 2–4 and Figure 1.

Table 2: T1: sequential per-operation latency in ms (5 runs × 60 measured iterations; one outstanding request). Mean ± 95% confidence interval of the run-level means; percentiles on the pooled sample. Endorse: proposal execution and endorsement by both organisations (includes chaincode execution); Submit: delivery to the ordering service; Commit: block cutting, validation and commit (BatchTimeout = 2 s).

Operation	N	Mean	P50	P95	P99	Endorse	Submit	Commit
State-changing								
Production registration	300	2044.6 ± 1.1	2044.4	2054.3	2060.7	13.2	3.3	2028.1
Transformation (mass balance)	300	2050.4 ± 1.4	2050.1	2059.4	2065.3	17.9	2.9	2029.6
Package creation	300	2045.0 ± 1.4	2044.8	2055.2	2060.7	14.1	2.7	2028.1
Binding registration (PDC)	300	2059.6 ± 1.9	2059.6	2069.3	2073.9	25.5	2.8	2031.4
Certificate consumption	300	2043.0 ± 1.1	2042.6	2051.3	2054.8	12.0	2.6	2028.5
Repeated scan (consumed cert.)	300	2047.8 ± 1.9	2047.7	2058.1	2068.0	16.3	2.8	2028.7
Read-only								
Production query	300	6.7 ± 0.1	6.8	8.2	9.2	–	–	–
Certificate query	300	6.4 ± 0.2	6.3	7.9	9.1	–	–	–
Certificate integrity check	300	5.4 ± 0.3	5.3	7.3	9.0	–	–	–
Authorization check	300	4.9 ± 0.3	4.7	6.8	7.7	–	–	–
UID verification (PDC)	300	10.2 ± 0.9	9.8	14.2	16.5	–	–	–

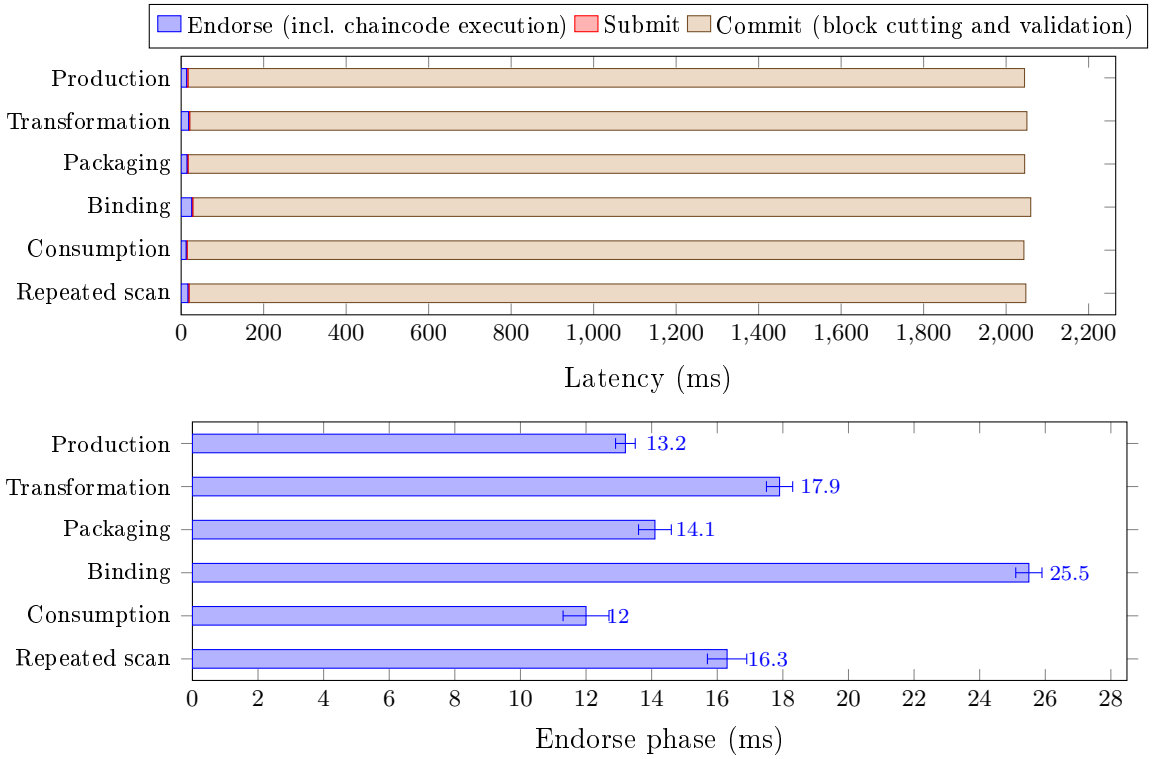


Figure 1: T1: decomposition of the latency of state-changing operations (top) and detail of the endorsement phase, which contains the chaincode execution (bottom; mean ± 95% CI over runs).

Table 3: T1: container resource usage during the measurement window (docker stats, ≈1 s sampling). CPU in % of one core; mean ± SD of the run-level means; maximum over all samples.

Container	CPU mean (%)	CPU max (%)	Memory (MiB)
Peer (Farmers)	1.87 ± 0.13	7.87	125.9 ± 3.5
Peer (Producers)	1.94 ± 0.09	6.39	119.4 ± 6.0
Orderer	0.31 ± 0.01	5.92	58.4 ± 10.4
certificate CC (Farmers)	0.33 ± 0.03	3.08	55.1 ± 0.4
certificate CC (Producers)	0.40 ± 0.05	4.02	54.1 ± 0.4
userauth CC (Farmers)	0.16 ± 0.03	4.07	53.4 ± 0.5
userauth CC (Producers)	0.18 ± 0.02	3.97	54.2 ± 0.7

Table 4: T1: persistent storage growth per complete workflow (production, transformation, packaging, binding, consumption and one repeated scan), in KB, measured per peer; mean ± SD over runs (`du`, 4 KB granularity).

Component	KB per workflow
Block files (peer)	49.36 ± 0.06
World state (LevelDB)	11.76 ± 0.03
Private data store	0.69 ± 0.05
History database	2.16 ± 0.00
Block files (orderer)	47.30 ± 0.00
Total per peer	63.96

Observations.

- **Block cutting dominates write latency.** The commit phase (2028–2031 ms) accounts for 98.6–99.3% of the end-to-end latency of every state-changing operation, consistently with `BatchTimeout` = 2 s and one transaction per block under sequential submission. Submission to the ordering service takes 2.6–3.3 ms.
- **Chaincode cost.** The endorsement phase, which contains the chaincode execution and the `userauth` authorization call, takes 12.0–25.5 ms. Relative to certificate consumption (12.0 ms), mass-balance transformation adds 5.9 ms and binding registration (HMAC commitment, certificate construction and private data dissemination with `requiredPeerCount`=1) adds 13.5 ms.
- **Read-only operations** complete in 4.9–10.2 ms on average (P99 $\leq$ 16.5 ms); UID verification is the most expensive query because it reads the private collection and re-verifies the whole binding chain.
- **Repeatability.** The between-run coefficient of variation of the mean latency is $\leq 0.08\%$ for state-changing and $\leq 6.7\%$ for read-only operations.
- **Resources.** Under sequential load each peer uses on average less than 2% of one CPU core (maximum 7.9%) and about 120 MiB of memory; the orderer uses 0.3%.
- **Storage.** One complete workflow (six transactions) adds 64.0 KB per peer, 77% of which are block files. Because each block contains a single transaction in this sequential setting, this value is an upper bound of the per-transaction storage cost; the batched case is measured in T2.

6.2 T3 – Concurrent conflicting operations

Executed on 2026-10-20 (16:10–16:20 CEST): 4 sub-tests, $k \in \{2, 5, 10, 20\}$ concurrent clients (one gRPC connection each), 10 targets per k and repetition, $R = 3$ repetitions, i.e. 480 targets and 4440 conflicting transactions. For each target the k requests were released together (median release spread 2.7 ms, maximum 11.7 ms); the expected outcome was declared before execution in the T3 design above.

Asset preparation. The 4290 preparation transactions (productions, transformations, packages and bindings) were executed by 40 parallel workflows in 23.6 s (181.7 valid transactions per second). 29 endorsements (0.67%) failed with `ERROR_INPUT_NOT_FOUND` or `ERROR_OUTPUT_LOT_NOT_FOUND` and succeeded when retried: a chained step was endorsed by a peer that had not yet committed the block containing the previous step, because the client observes the commit status only on its gateway peer.

Table 5: T3: outcomes of concurrent conflicting operations. *Exactly 1*: targets with exactly one committed state transition; *MVCC*: requests invalidated at commit with `MVCC_READ_CONFLICT`; *Rejected*: requests rejected by the chaincode at endorsement or committed as repeated scans; *Other*: infrastructure errors; *Post-check*: targets whose final ledger state matched the expected state; *Violations*: targets with more than one committed transition or an inconsistent final state.

Configuration	Targets	Requests	Exactly 1	MVCC	Rejected	Other	Post-check	Violations
T3-a: Same UID, k concurrent `ConsumeByUID` – Single consumption (Eq. 16)								
$k = 2$	30	60	30	30	0	0	30/30	0
$k = 5$	30	150	30	120	0	0	30/30	0
$k = 10$	30	300	30	270	0	0	30/30	0
$k = 20$	30	600	30	570	0	0	30/30	0
T3-b1: Same UID, k different packages – Active UID-binding uniqueness (Eq. 13)								
$k = 2$	30	60	30	30	0	0	30/30	0
$k = 5$	30	150	30	120	0	0	30/30	0
$k = 10$	30	300	30	270	0	0	30/30	0
$k = 20$	30	600	30	570	0	0	30/30	0
T3-b2: k different UIDs, same package – Active output-lot binding uniqueness (Eq. 14)								
$k = 2$	30	60	30	30	0	0	30/30	0
$k = 5$	30	150	30	120	0	0	30/30	0
$k = 10$	30	300	30	270	0	0	30/30	0
$k = 20$	30	600	30	570	0	0	30/30	0
T3-c: Same input lot, k transformations – Input-consumption integrity								
$k = 2$	30	60	30	30	0	0	30/30	0
$k = 5$	30	150	30	120	0	0	30/30	0
$k = 10$	30	300	30	270	0	0	30/30	0
$k = 20$	30	600	30	570	0	0	30/30	0
Total	480	4440	480	3960	0	0	480/480	0

Observations.

- **No invariant violation.** In all 480 targets at most one conflicting request produced a committed state transition (0 violations); 480 targets had exactly one and none ended without a commit. The final ledger state matched the expected state in 480 of 480 targets.
- **Enforcement mechanism.** Because the k requests were released together, they were in most cases endorsed before the winning transaction was committed and therefore read the same state version: 3960 of the 3960 losing requests were invalidated by Fabric MVCC validation at commit time, while 0 were endorsed after the winning commit and were rejected by the chaincode preconditions (or, for T3-a, recorded as repeated scans). Single consumption, binding uniqueness and input-consumption integrity are therefore enforced jointly by the chaincode preconditions (for sequential requests) and by MVCC validation (for concurrent requests), as argued in the proof sketches.
- **Effect of block cutting.** The latency of the winning transaction was 2040–2077 ms for $k \in \{2, 5\}$ and 59–131 ms for $k \in \{10, 20\}$: with ten or more requests released together the block reaches `MaxMessageCount`=10 and is cut immediately instead of waiting for the 2 s `BatchTimeout`. This confirms that the sequential write latency measured in T1 is determined by the block-cutting configuration rather than by chaincode execution.
- **Scope.** The experiment verifies the digital invariants under concurrent submission by authorised clients. It does not test physical tag cloning itself, malicious endorsing peers or compromised organisations, which remain outside the evaluated threat classes.

6.3 T1-b – Effect of the block-cutting timeout

The T1 procedure was repeated after changing only the channel parameter `BatchTimeout` through a signed channel configuration update (all other parameters unchanged): 1 s (5 runs × 30 iterations, 16:25–16:42 CEST) and 0.5 s (5 runs × 30 iterations, 16:42–16:51 CEST), each with 2 excluded warm-up iterations. The 2 s values are those of T1. All 3300 additional requests completed successfully (0 failures). The original value of 2 s was restored at the end of the experiment.

Table 6: T1-b: mean latency (ms) of each operation for `BatchTimeout` = 2 s, 1 s and 0.5 s. End-to-end latency and endorsement phase (chaincode execution) of state-changing operations; end-to-end latency of read-only queries.

	End-to-end			Endorse		
Operation	2 s	1 s	0.5 s	2 s	1 s	0.5 s
State-changing						
Production registration	2044.6	1040.3	531.2	13.2	12.6	7.9
Transformation (mass balance)	2050.4	1044.1	534.6	17.9	16.2	11.1
Package creation	2045.0	1041.1	532.4	14.1	13.0	8.9
Binding registration (PDC)	2059.6	1052.7	542.4	25.5	22.9	18.1
Certificate consumption	2043.0	1038.0	532.5	12.0	10.6	8.5
Repeated scan (consumed cert.)	2047.8	1042.0	532.1	16.3	14.3	9.9
Read-only						
Production query	6.7	6.2	4.2		–	
Certificate query	6.4	5.6	4.3		–	
Certificate integrity check	5.4	4.8	3.7		–	
Authorization check	4.9	4.4	3.4		–	
UID verification (PDC)	10.2	8.9	7.1		–	

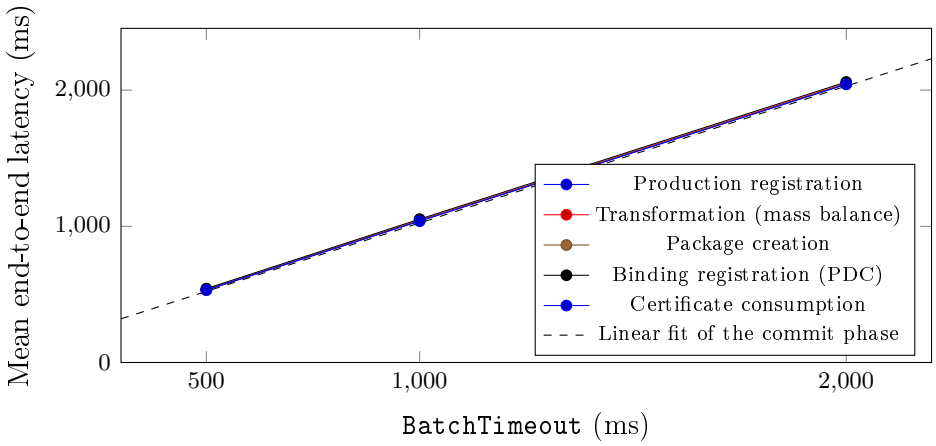


Figure 2: T1-b: end-to-end latency of state-changing operations as a function of `BatchTimeout`.

Observations.

- **Causal confirmation.** The mean commit-phase latency of state-changing operations follows $L^{\text{com}} \approx 19.3\,\text{ms} + 1.005 \cdot \texttt{BatchTimeout}$ ($R^2 = 1.0000$): reducing the timeout from 2 s to 0.5 s reduces the end-to-end latency of every write by the same amount, which confirms that under sequential submission the write latency is determined by the block-cutting configuration of the ordering service.
- **Chaincode cost does not depend on the ordering configuration.** The endorsement phase does not increase when the timeout is reduced: it ranges from 12.0–25.5 ms at 2 s to

7.9–18.1 ms at 0.5 s. Read-only queries, which do not involve the ordering service, show a similar reduction (from 4.9–10.2 ms to 3.4–7.1 ms). Because both endorsement and queries became faster, the difference is attributed to the execution environment rather than to the chaincode, most plausibly to CPU power management on the host (shorter timeouts reduce the idle time between consecutive requests); this interpretation was not verified separately. In all configurations the Enhanced SPECTRA mechanisms account for at most about 25 ms per transaction.

- **Deployment implication.** `BatchTimeout` is a deployment trade-off: a shorter timeout lowers the latency of individual transactions under light load, at the cost of smaller blocks and higher per-transaction block overhead. Throughput under load is evaluated in T2 with the default value of 2 s.

6.4 T2 – Offered load and saturation

Executed on 2026-10-20 (17:05–17:47 CEST) with the benchmark harness acting as an open-loop load generator (fixed inter-arrival times, 32 client connections per identity; requests are released on schedule without waiting for previous responses). Offered rates 10, 20, 40, 80, 160, 320, 640, 1280 TPS; for each rate 3 repetitions of 15 s warm-up, 30 s measurement window and up to 30 s drain. Only requests *submitted* in the measurement window are counted; throughput and goodput are averages over the window. The state-changing workload is `RegisterProduction` (independent keys, includes the `userauth` authorization call, no prepared assets required); the read-only workload is `VerifyByUID` on 50 prepared bound UIDs. A workload sweep stops after two consecutive non-sustainable rates. `BatchTimeout` = 2 s, `MaxMessageCount` = 10. Intermediate rates were added in a second session on 2026-10-21 (write: 160, 200, 240 TPS, 08:23–08:31 CEST; read: 320, 360, 400 TPS, 08:43–08:50 CEST) on a freshly re-created network with the same configuration, using the P95 reference of the first session (write 946.3 ms, read 18.4 ms) for the saturation criterion. One rate per workload was repeated in both sessions as an anchor: state-changing 160 TPS, goodput 160.0 vs 160.0 TPS, P95 110.7 vs 352.7 ms; read-only 320 TPS, goodput 320.0 vs 320.0 TPS, P95 16.9 vs 14.7 ms (first vs second session).

Table 7: T2: offered load sweep (mean over 3 repetitions; goodput ± 95% CI). λ: measured submission rate; goodput: valid transactions per second; latency percentiles of valid requests; CPU: mean of the two peers and of the orderer in % of one core during the measurement window; sustainable: saturation criterion of Section 2.5 satisfied in every repetition.

Offered (TPS)	λ (TPS)	Goodput (TPS)	P50 (ms)	P95 (ms)	P99 (ms)	Fail (%)	CPU peer (%)	CPU ord. (%)	Sust.
State-changing (`RegisterProduction`)									
10	10.0	10.0 ± 0.0	497.4	946.3	950.2	0.00	9.3	2.0	yes
20	20.0	20.0 ± 0.0	258.5	479.9	487.1	0.00	11.6	2.3	yes
40	40.0	40.0 ± 0.0	147.3	255.5	260.0	0.00	20.8	5.0	yes
80	80.0	80.0 ± 0.0	89.0	143.9	147.3	0.00	44.0	10.6	yes
160	160.0	160.0 ± 0.0	78.7	110.7	120.7	0.00	129.1	34.6	yes
200	200.3	200.3 ± 1.4	4584.3	5832.4	5965.9	0.00	153.7	46.3	no
240	240.0	240.0 ± 8.9	22896.6	31719.3	32664.3	0.00	141.9	39.2	no
320	318.1	318.1 ± 12.5	32410.6	39914.7	40969.1	0.00	136.8	34.7	no
640	651.3	0.5 ± 2.1	49502.2	56722.1	56886.1	99.93	170.2	13.2	no
Read-only (`VerifyByUID`)									
10	10.0	10.0 ± 0.0	14.2	18.4	21.3	0.00	4.8	0.1	yes
20	20.0	20.0 ± 0.0	15.7	19.0	22.1	0.00	9.2	0.1	yes
40	40.0	40.0 ± 0.0	11.6	15.2	16.9	0.00	14.5	0.1	yes
80	80.0	80.0 ± 0.0	6.6	9.0	11.1	0.00	16.3	0.0	yes
160	160.0	160.0 ± 0.0	7.2	9.1	11.0	0.00	31.7	0.1	yes
320	320.0	320.0 ± 0.0	10.9	16.9	24.0	0.00	64.6	0.1	yes
360	360.0	360.0 ± 0.0	13.4	71.0	131.2	0.00	69.2	0.1	no
400	400.0	400.0 ± 0.0	2185.7	4269.4	4382.8	0.00	82.3	0.1	no
640	640.0	0.0 ± 0.0	–	–	–	100.00	111.5	0.1	no
1280	1149.0	0.0 ± 0.0	–	–	–	100.00	88.6	0.1	no

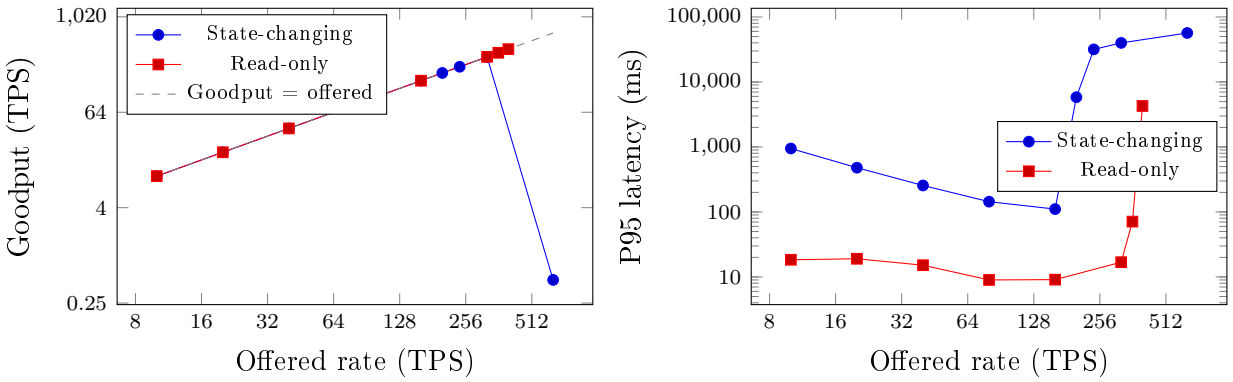


Figure 3: T2: goodput (left) and P95 latency (right) as a function of the offered rate (logarithmic axes).

Observations.

- **State-changing workload:** the highest sustainable offered rate was 160 TPS (goodput 160.0 TPS, P95 110.7 ms); the criterion was first violated at 200 TPS (P95>2*base, backlog growing), where the goodput reached 200.3 TPS.
- **Read-only workload:** the highest sustainable offered rate was 320 TPS (goodput 320.0 TPS, P95 16.9 ms); the criterion was first violated at 360 TPS (P95>2*base), where the goodput reached 360.0 TPS.
- **Latency under load.** For state-changing transactions the latency first *decreases* with the offered rate, because blocks reach `MaxMessageCount` = 10 before the 2 s `BatchTimeout` expires

(T1-b), and increases again only when the system approaches saturation.

- **Storage under load.** The 93983 valid `RegisterProduction` transactions of the sweep added 548.0 MB of block files, 27.7 MB of world state and 6.1 MB of history database per peer, i.e. 6.34 KB per transaction (5.97 KB of block files) with batched blocks.
- **Behaviour beyond saturation.** Above the saturation point the latency grows without bound (tens of seconds at 200–320 TPS for writes and 400 TPS for reads); goodput values reported for these rates count requests that were eventually committed within the drain window and do *not* represent sustainable throughput. At 640 TPS (writes) and 640–1280 TPS (reads) practically all requests exceeded the client deadlines (10 s for queries, 30 s for endorsement and submission), so the measured goodput dropped to zero. The cause was verified in the second session, where failure classes include the gRPC status: all 12,000 failed queries of a read run at 400 TPS executed under a residual backlog were `DEADLINE_EXCEEDED`, i.e. no request was rejected by the peers or by the chaincode.
- **Session comparability and excluded series.** The anchor rates reproduced the first session: reads at 320 TPS gave P95 14.3–15.3 ms (16.9 ms in the first session); writes at 160 TPS gave P95 104 and 114 ms in two of three repetitions (110.7 ms in the first session), while one repetition experienced a transient backlog (P95 840 ms, maximum 223 outstanding requests) that was absorbed within the run and raises the mean reported above. A first read refinement started immediately after the 240 TPS write runs was discarded, because the peers were still validating the backlog of previously submitted transactions (P95 at 320 TPS between 44 and 2171 ms); it was repeated after verifying that the peers were idle (CPU below 2%).
- **Resource utilisation at saturation.** At the state-changing saturation point each peer used on average 1.3–1.5 CPU cores out of the 22 available to WSL2, and the orderer about 0.4 cores; the host CPU was therefore not the limiting resource. The most plausible limiting factor is the sequential validation and commit of small blocks (10 transactions each, about 16–20 blocks per second at saturation); this hypothesis was not verified separately.
- **Scope.** The load generator runs on the same host as the network; the reported capacity is therefore a lower bound for the tested single-host configuration and not a property of Fabric or of Enhanced SPECTRA in general.

6.5 T4 – Transactional bursts

Executed on 2026-10-21 (09:05–09:26 CEST) with the state-changing operation `RegisterProduction` and 32 client connections. Submission schedules were precomputed and saved with their seed (20261021); requests are released open-loop at their scheduled offsets. Profile A releases B requests at the same instant; profile B releases 50 requests every 5 s for 60 s (inter-burst interval 5 s, burst duration ≈0 s); profile C draws exponential inter-arrival times with mean rates 80 and 128 TPS over 60 s (approximately 50% and 80% of the maximum sustainable state-changing rate measured in T2). Each configuration was repeated 5 times with 10 s of idle time between runs. Completion time is measured from the first submission to the last commit of the run.

Table 8: T4: burst and Poisson workloads (mean over 5 repetitions; completion time ± 95% CI). Requests: per run; Valid: over all repetitions; latency percentiles of valid transactions; Max out.: maximum number of outstanding requests; Blocks: blocks used per run.

Profile	Requests	Valid	Completion (s)	P50 (ms)	P95 (ms)	P99 (ms)	Max out.	Blocks
A: single burst, $B = 10$	10	50/50	0.09 ± 0.02	85	90	90	10	1.0
A: single burst, $B = 50$	50	250/250	0.27 ± 0.03	236	260	264	50	5.0
A: single burst, $B = 100$	100	500/500	0.43 ± 0.03	380	418	424	100	10.0
A: single burst, $B = 200$	200	1000/1000	0.80 ± 0.05	670	749	776	200	20.0
B: 50 every 5 s for 60 s	600	3000/3000	55.22 ± 0.02	182	216	229	50	60.0
C: Poisson, $\bar{\lambda} = 80$ TPS	4778	23891/23891	61.96 ± 0.06	87	183	227	22	478.2
C: Poisson, $\bar{\lambda} = 128$ TPS	7684	38422/38422	62.00 ± 0.02	75	131	160	43	768.8

Observations.

- **Correctness under bursts.** 67113 of 67113 requests were committed as valid (0 invalid or unresolved).
- **Burst absorption.** Single bursts of 10, 50, 100, 200 requests (released within at most 118.7 ms) were completed in 0.09 s, 0.27 s, 0.43 s, 0.80 s respectively, from the first submission to the last commit, with P95 latencies of 90, 260, 418, 749 ms. In the periodic profile, 60 of 60 bursts were fully committed before the next burst was released (maximum burst completion time 0.28 s), so no backlog accumulated between bursts.
- **Random arrivals.** Under Poisson arrivals (79.6 TPS achieved, inter-arrival CV 1.00; 128.1 TPS achieved, inter-arrival CV 0.99) the P95 latency was 183 ms and 131 ms and the maximum number of outstanding requests was 22 and 43 respectively.
- **Bursts above the sustainable rate.** The largest single burst (200 requests) was committed in 0.80 s, i.e. at about 250 TPS, above the maximum sustainable rate of 160 TPS measured in T2: short peaks above capacity are buffered and absorbed, provided that the average load remains below it. In profiles B and C all blocks were cut by size (on average 10.0 transactions per block), so even at 80 TPS no block waited for the 2 s `BatchTimeout`.
- **Release spread.** The protocol planned single bursts released within 100 ms. The observed release spread, determined by the client-side signing of the proposals, was 13.6, 45.7, 70.1 and 118.7 ms for $B = 10$, 50, 100 and 200; the 200-request burst therefore slightly exceeded the planned value.
- **Batching effect.** Larger bursts fill blocks of `MaxMessageCount` = 10 transactions immediately, so the latency of a burst is driven by the number of blocks to be cut and committed rather than by the 2 s `BatchTimeout`.

6.6 T5 – Fault injection and recovery

Executed on 2026-10-21 (09:34–10:17 CEST). Two concurrent open-loop streams were submitted through the FarmersMSP gateway peer: `RegisterProduction` at 32 TPS and `VerifyByUID` at 64 TPS (20% of the maximum sustainable rates measured in T2). Each run consisted of 60 s of normal operation, a 30 s fault window and 90 s of observation after restoration; each fault was repeated 3 times. In the test-network each peer is attached to two Docker networks; the first F4 series disconnected peer0.producers from only one of them (reported as F4p), and F4 was therefore repeated disconnecting the peer from all its networks (10:34–10:46 CEST). After each run, every submitted write was reconciled against the ledger of both peers (`qscc GetTransactionByID`), every committed lot was read back from both peers, and the block heights of the two peers were compared.

Table 9: T5: availability and recovery under injected faults (mean over 3 runs). *Gap*: longest interval without a successful request in the run; *Recovery*: time from restoration to the first successful request submitted after it; *OK in fault*: share of the requests submitted during the fault window that eventually succeeded; *Max lat.*: maximum end-to-end latency of the successful writes submitted during the fault window (baseline P95 before the fault: F1 0.3 s, F2 0.3 s, F3 0.3 s, F4p 0.3 s, F4 0.3 s); *Consistent*: runs in which both peers converged to the same height, every client-confirmed write was committed as valid on both peers, no failed endorsement was committed and every committed lot was readable on both peers.

	State-changing				Read-only		
Fault	Gap (s)	Recovery (s)	OK in fault (%)	Max lat. (s)	Gap (s)	OK in fault (%)	Consistent
F1: Peer (Producers) stopped	31.0	1.0	0.0	–	1.6	100.0	3/3
F2: Orderer stopped	44.6	14.5	0.0	–	1.0	100.0	3/3
F3: Chaincode container killed	0.5	0.0	100.0	2.8	0.5	100.0	3/3
F4p: Peer (Producers): one of two networks removed	14.8	0.5	51.9	1.0	0.9	100.0	3/3
F4: Peer (Producers) fully partitioned	33.0	3.0	0.0	–	1.5	100.0	3/3

Observations.

- **F1 (peer (producers) stopped).** Expected: writes unavailable (endorsement policy requires both organisations); reads through the Farmers peer unaffected. Observed: longest write gap 31.0 s, writes recovered 1.0 s after restoration (dominant write failures: `ENDORSE:GRPC_UNAVAILABLE (2513), ENDORSE:GRPC_ABORTED (461)`); longest read gap 1.6 s (read failures: `none`). Of the writes submitted during the fault window, none eventually succeeded (0% in each run).
- **F2 (orderer stopped).** Expected: writes cannot be ordered; reads unaffected. Observed: longest write gap 44.6 s, writes recovered 14.5 s after restoration (dominant write failures: `SUBMIT:GRPC_UNAVAILABLE (4276)`); longest read gap 1.0 s (read failures: `none`). Of the writes submitted during the fault window, none eventually succeeded (0% in each run).
- **F3 (chaincode container killed).** Expected: requests needing the killed chaincode container fail or are delayed until the peer relaunches it. Observed: longest write gap 0.5 s, writes recovered 0.0 s after restoration (dominant write failures: `none`); longest read gap 0.5 s (read failures: `none`). Of the writes submitted during the fault window, 100.0% eventually succeeded (per run: 100, 100, 100%), with a maximum latency of 2.8 s, i.e. they were delayed until the component became available again rather than rejected.
- **F4p (peer (producers): one of two networks removed).** Expected: writes interrupted until the peers re-establish their connection through the remaining network. Observed: longest write gap 14.8 s, writes recovered 0.5 s after restoration (dominant write failures: `ENDORSE:GRPC_ABORTED (1388)`); longest read gap 0.9 s (read failures: `none`). Of the writes submitted during the fault window, 51.9% eventually succeeded (per run: 28, 100, 28%), with a maximum latency of 1.0 s, with normal latency: they were not delayed but endorsed through the remaining network path.
- **F4p in detail.** In the test-network each peer is attached to two Docker networks. In two of the three F4p runs all writes failed from the disconnection until about 21 s later, when the gateway re-established the connection to peer0.producers through the remaining network and writes resumed while the peer was still disconnected from the first one; in the third run

the active connection already used the remaining network and no write failed. F4p therefore shows automatic fail-over to a redundant network path; the full partition is F4.

- **F4 (peer (producers) fully partitioned).** Expected: as F1, without stopping the peer process. Observed: longest write gap 33.0 s, writes recovered 3.0 s after restoration (dominant write failures: `ENDORSE:GRPC_ABORTED (2275), ENDORSE:GRPC_UNAVAILABLE (893)`); longest read gap 1.5 s (read failures: `none`). Of the writes submitted during the fault window, none eventually succeeded (0% in each run).
- **State after recovery.** In 15 of 15 runs the ledger was consistent after recovery: 74594 of 74594 client-confirmed writes were committed as valid on both peers, 0 writes with an unknown client outcome were found committed and 4276 were not committed, 0 discrepancies were found, and 0 of 149188 lot read-backs failed. No transaction was lost after confirmation, and no rejected transaction was committed.
- **Scope.** With one peer per organisation, an endorsement policy requiring both organisations and a single-node ordering service, F1, F2 and F4 necessarily interrupt state-changing operations; the experiment characterises the availability and the recovery of the tested topology, not the fault tolerance of Fabric deployments with redundant peers or a multi-node Raft cluster.

6.7 T6 – Sensitivity to inter-node network delay (analytical estimate)

Network emulation with `tc netem` was not possible on the evaluation host, because the WSL2 kernel does not provide the `netem` queueing discipline (Step 0). Instead of an approximate emulation, the effect of an inter-organisation round-trip time (RTT) is *estimated* from the latency decomposition measured in T1 and T2, counting the network exchanges on the critical path of a Fabric Gateway transaction. Assumptions: each organisation runs its peer and its clients in the same site (client–gateway delay negligible); organisations and the ordering service are in different sites separated by the same RTT; gRPC connections are persistent (no TLS handshake per transaction); bandwidth, packet loss and jitter are not modelled.

- State-changing transaction: endorsement by the remote organisation (1 RTT, the local endorsement runs in parallel), submission to the ordering service (1 RTT) and delivery of the block from the orderer to the gateway peer (0.5 RTT), i.e. $\Delta L \approx 2.5\,\mathrm{RTT}$. Binding registration requires in addition the dissemination of the private data to one other member peer before the endorsement is returned (`requiredPeerCount`=1), i.e. $\Delta L \approx 3.5\,\mathrm{RTT}$. A multi-node Raft ordering service would add the consensus round among orderers.
- Read-only query: evaluated on the gateway peer of the client's organisation, $\Delta L \approx 0$; about 1 RTT if the client is remote from its peer.

Table 10: T6: estimated latency (ms) for an inter-site RTT of 20, 50 and 100 ms, obtained by adding the critical-path network delay to the values measured on the single host (RTT $\approx$ 0). Analytical estimate, not a measurement.

Operation	ΔL	Measured	RTT 20	RTT 50	RTT 100
Production registration, sequential (T1 mean)	2.5 RTT	2044.6	2094.6	2169.6	2294.6
Certificate consumption, sequential (T1 mean)	2.5 RTT	2043.0	2093.0	2168.0	2293.0
Binding registration, sequential (T1 mean)	3.5 RTT	2059.6	2129.6	2234.6	2409.6
State-changing at 160 TPS (T2 P50)	2.5 RTT	78.7	128.7	203.7	328.7
State-changing at 160 TPS (T2 P95)	2.5 RTT	110.7	160.7	235.7	360.7
UID verification, client at its peer (T1 mean)	0	10.2	10.2	10.2	10.2
UID verification, remote client (T1 mean)	1 RTT	10.2	30.2	60.2	110.2

Observations.

- Under sequential submission, an RTT of 100 ms would increase write latency by about 12% (from about 2.04 to 2.29–2.41 s), because the 2 s block-cutting timeout remains the dominant term (T1, T1-b).
- Under load, where blocks are cut by size, the network delay becomes the largest latency component: the median write latency at 160 TPS would grow from 79 to about 330 ms at RTT 100 ms.
- Since the transaction flow is pipelined, a higher RTT increases the number of in-flight transactions (by Little's law, about 40 additional outstanding transactions at 160 TPS and RTT 100 ms) rather than directly reducing throughput; this expectation was not verified experimentally.
- These figures are estimates under the stated assumptions. A geographically distributed deployment remains a limitation of the evaluation (Section 5).

6.8 T7 – Comparison with the original SPECTRA

An experimental comparison with the original SPECTRA implementation was not performed. Both implementations run on the same Fabric transaction flow, in which the latency of state-changing operations is dominated by block cutting (T1, T1-b); a direct end-to-end comparison would therefore differ only by the chaincode execution time, which is isolated in the endorsement phase measured in T1. The incremental contribution of Enhanced SPECTRA is reported per mechanism in Table 11, together with its measured cost.

Table 11: T7: functional comparison between the original SPECTRA [14] and Enhanced SPECTRA, with the measured cost of each mechanism (T1: mean endorsement phase of state-changing operations, which contains the chaincode execution, or mean end-to-end latency of queries; T3: concurrency experiment).

Mechanism	SPECTRA	Enhanced	Measured cost / evidence
Selective verification with private data collections	yes	yes	UID verification query 10.2 ms
NFC–certificate–batch cryptographic binding	yes	yes	binding registration: endorsement 25.5 ms
Keyed UID commitment, active UID and output-lot binding uniqueness, audited re-binding	no	yes	included in binding registration; uniqueness under concurrency: 0 violations in 240 targets (T3-b)
Mass-balance validation of transformations	no	yes	transformation: endorsement 17.9 ms (+5.9 ms w.r.t. consumption)
On-chain governance (registration, roles, revocation, audit)	no (application layer)	yes	authorization check 4.9 ms as a query (upper bound of the call performed inside every protected write)
One-shot certificate consumption with repeated-scan logging	no	yes	consumption: endorsement 12.0 ms; repeated scan 16.3 ms; single consumption under concurrency: 0 violations in 120 targets (T3-a)
Input-consumption integrity of certified lots	no	yes	0 violations in 120 targets (T3-c)

Observation. The chaincode execution of every operation, including the mechanisms introduced by Enhanced SPECTRA, takes at most about 25 ms per transaction, about 80 times less than the 2 s block-cutting timeout that dominates sequential write latency, while its invariants were preserved in all concurrency and fault-injection experiments.

6.9 T8 – Effect of supply-chain complexity

Executed on 21 October 2026 (11:38–12:25 CEST), sequential submission, on the network used for T2–T5 (ledger enlarged by the previous experiments). 816 measured transactions, 0 failures, 0 retries of a measured step.

Table 12: T8-A (breadth): mean chaincode execution (L^{end}, ms), UID verification latency (ms) and certificate size as a function of the number of input lots n (24 samples per n; 5 verifications per certificate).

n	Transformation	Packaging	Binding	Consumption	Verification	Certificate (B)
1	29.0	17.0	29.7	14.0	11.5	1124
5	39.9	17.1	28.8	12.3	9.8	1236
10	55.1	15.4	26.9	11.7	9.3	1373
25	93.1	16.4	28.1	11.1	8.9	1780
50	149.1	14.9	28.0	10.8	9.4	2457
100	257.0	13.0	27.2	11.1	8.5	3809

Table 13: T8-B (depth): mean chaincode execution (L^{end}, ms), UID verification latency (ms) and certificate size as a function of the number of successive transformations d (5 chains per d).

d	Transf. (mean)	Last transf.	Binding	Consumption	Verification	Certificate (B)
1	24.1	24.1	27.9	11.3	8.0	1135
5	19.3	18.5	23.4	10.5	8.0	1142
10	18.0	18.6	26.3	11.1	8.5	1142
20	18.2	19.3	23.5	10.0	9.5	1142

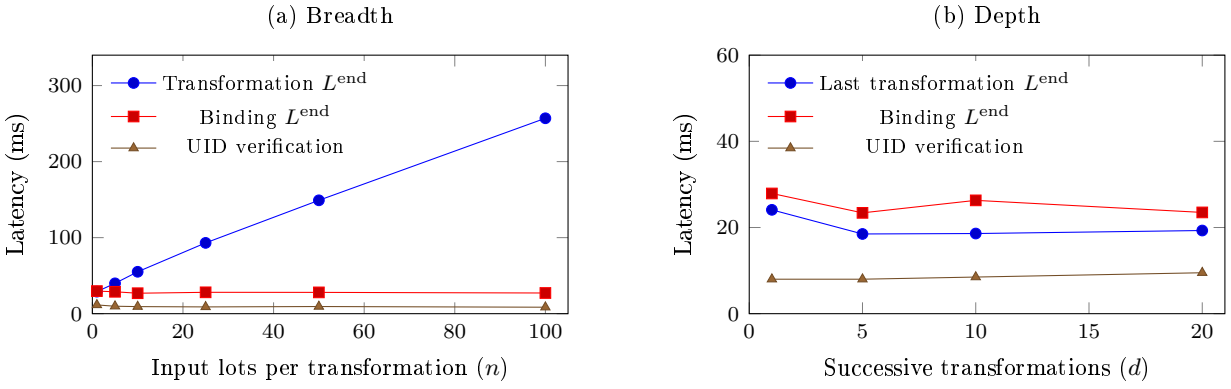


Figure 4: T8: cost as a function of supply-chain complexity: (a) one transformation with n input lots; (b) a chain of d successive transformations before packaging and binding.

Observations.

- **Breadth.** The chaincode execution of a transformation grows linearly with the number of inputs: $30.9 + 2.29\,n$ ms ($R^2 = 1.00$), i.e. 257 ms for 100 input lots, still about one eighth of

the 2 s block-cutting timeout; mass balance and input consumption read and update every input lot.

- **Downstream operations do not depend on breadth.** Binding stays at 27–30 ms (fit slope −0.01 ms per input) although the certificate, which embeds the list of inputs, grows from 1.1 to 3.8 KB; UID verification stays at 8.5–11.5 ms; packaging and consumption are constant.
- **Depth.** After 20 successive transformations, verification takes 9.5 ms against 8.0 ms after one (slope 0.08 ms per upstream transformation), and binding and certificate size do not change (slopes −0.16 ms and 0.3 B): the certificate commits to the hashes of its immediate inputs and transformation, so the lineage is not re-traversed at verification time. Each transformation of a chain costs 18–24 ms; the mass balance remained exact along the whole chain (10 kg reduced to about 3.6 kg after 20 steps of yield 0.95).
- **Absolute values.** The chaincode execution times are 10–60% higher than in T1 (e.g. 29.0 versus 17.9 ms for a single-input transformation), because T8 ran after T2–T5 on a ledger enlarged by those experiments; the effect of breadth and depth is therefore assessed only by comparisons within T8.
- **Scope.** The breadth expressible in one transaction is bounded by the implementation limit of 100 input lots and by the 16 KB payload limit of the chaincode.